

Assignment 4

Software Testing Report

Student	Perni Bharath Raghavendra
Roll number	150096724139
Course	SEM5 — Software Engineering & Project Management
Selected project	TravelEase — Travel Booking Workflow Simulation
Repository	https://github.com/bharath-541/travelease-system-design

Prepared from the project repository and assignment requirements

1. Test objective and scope

TravelEase is a Flask academic booking simulation. The tested workflow covers search, dynamic pricing, transactional inventory holds, payment outcomes, provider confirmation, cancellation and refund compensation. The goal of this report is to show that the core state transitions preserve inventory and payment consistency under both success and failure paths.

In-scope behaviours

- Search filters and cache hits for a Mumbai-to-Delhi flight query.
- Successful booking confirmation after a payment success response.
- Payment failure releasing a temporary inventory hold.
- Provider timeout compensating both payment and inventory.
- Cancellation restoring inventory and refunding payment.
- Rendering of the home and search result pages.

Out of scope

- Live payment gateways, live travel-provider integrations and production load testing.
- Penetration testing, multi-region failover and long-running reconciliation jobs.

2. Test plan summary

Item	Plan
Test level	Unit and integration-style workflow tests through pytest; one web rendering check.
Test data	Seeded SQLite listings, including FLT-BOM-DEL-101 and a two-traveller hold.
Environment	Python virtual environment with Flask, Pillow, ReportLab and pytest.
Oracle	HTTP 200 responses, expected booking status, payment status and inventory deltas.
Regression rule	Run the complete suite after workflow or data-model changes; all six tests must pass.

3. Executed test cases

ID	Scenario	Expected result	Observed result
TC-01	Search filters and cache	First query is a miss; repeat query is a hit; results contain a positive quote.	Passed
TC-02	Successful booking	Payment success produces CONFIRMED booking and provider confirmation.	Passed
TC-03	Payment failure	Booking becomes PAYMENT_FAILED and the hold is released.	Passed
TC-04	Provider timeout	Booking becomes PROVIDER_FAILED; payment is REFUNDED; inventory is restored.	Passed
TC-05	Cancellation	Confirmed booking becomes CANCELLED; payment is REFUNDED; inventory is restored.	Passed
TC-06	Web pages render	Home and search pages return HTTP 200 and expected content.	Passed

Execution result: 6 passed in 0.16 seconds using pytest. No confirmed defect was observed in this executed suite; the remaining items in this report are coverage gaps and recommended tests.

4. Testing techniques

4.1 Equivalence partitioning

Input or state	Valid partition	Invalid or alternate partition
Payment result	success	payment_failure, provider_timeout, provider_rejection
Travel type	flight, hotel, bus, train or package	unsupported or missing type
Inventory	available seats or rooms $\geq$ travellers	insufficient inventory or expired hold
Search cache	same normalised query repeated	different route, date or filter
Booking state	HELD $\rightarrow$ CONFIRMED $\rightarrow$ CANCELLED	invalid transition or repeated cancellation

4.2 Boundary value analysis

Boundary	Boundary tests to add
Travellers	0, 1, maximum available inventory, maximum + 1.
Hold lifetime	Just before expiry, at expiry and just after expiry.
Inventory	Exact inventory match, one below required and zero inventory.
Price	Minimum valid amount, zero, negative amount and large amount.
Cache	First request, immediate repeat, expired entry and changed query.

4.3 Failure-path assertions

TravelEase is strongest where a failure must undo earlier work. The tests assert both the public booking status and the underlying payment or inventory state. This matters because a page-level “failure” message alone would not prove that overbooking or duplicate charges were prevented.

5. White-box and regression analysis

Component or rule	White-box check	Regression value
BookingService.create_hold	Verify inventory decreases by travellers and a hold record is written.	Protects the reservation invariant.
PaymentService.process	Exercise each simulated provider result and inspect state transitions.	Protects compensation behaviour.
CancellationService.cancel	Verify only eligible bookings can be cancelled and inventory is restored once.	Prevents duplicate refunds or releases.
SearchService.search	Verify filter predicate and cache-key normalisation.	Protects result correctness and cache behaviour.
Flask routes	Assert status code and representative response text.	Catches template or routing regressions.

6. Evidence from the running application

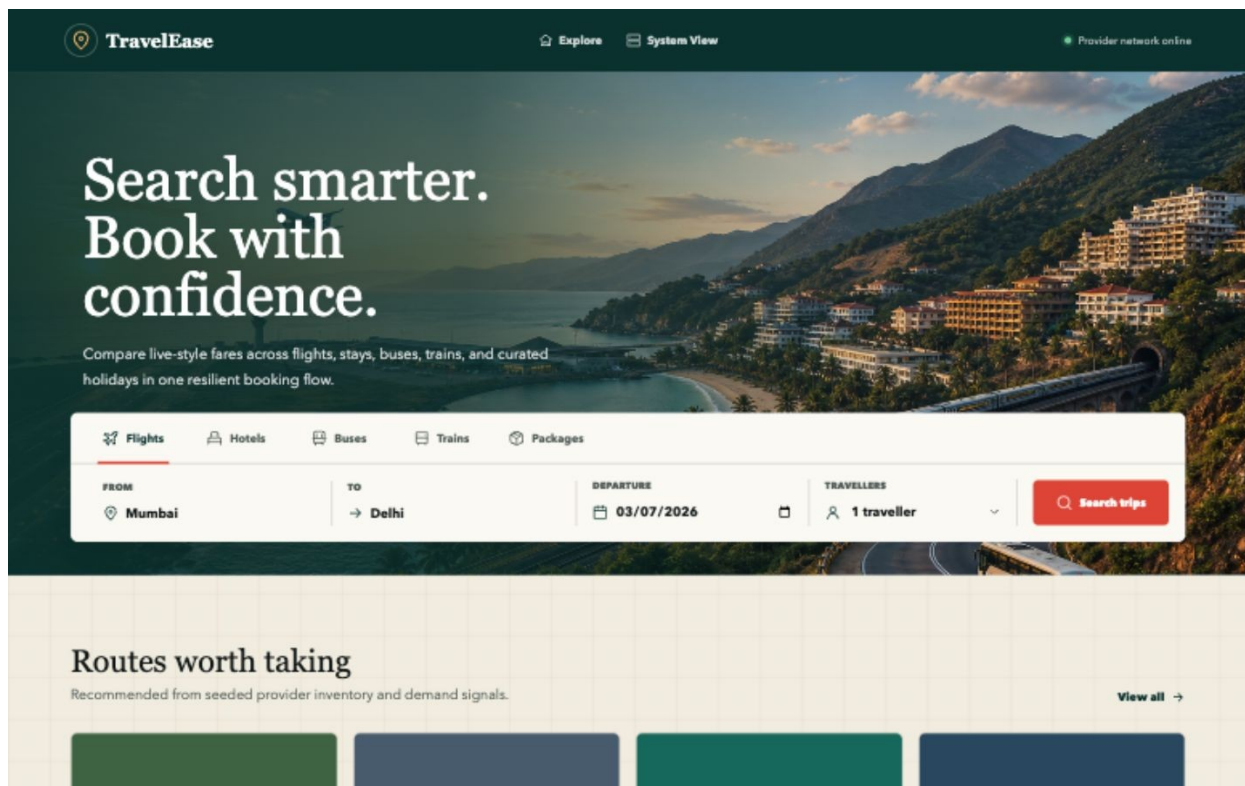


Figure 1. TravelEase search landing page from the repository screenshots.

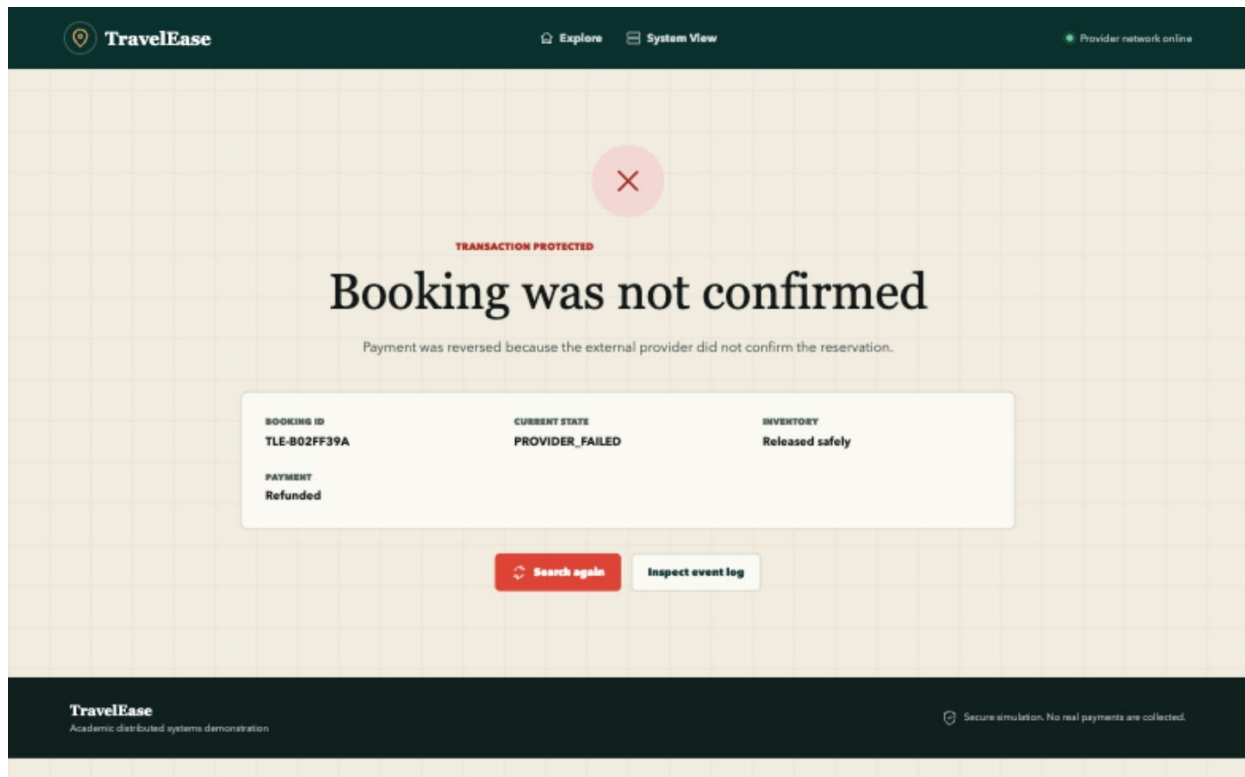


Figure 2. Failure-state screenshot used as visual evidence for compensation flows.

7. Test summary and coverage gaps

Area	Status	Next test or control
Core workflow transitions	Covered by six passing tests.	Add property-based checks for repeated and out-of-order events.
Search and caching	One happy path and repeat query covered.	Add invalid filters, cache expiry and concurrent requests.
Web rendering	Home and search pages covered.	Add booking review, payment and cancellation page assertions.
Persistence	SQLite state inspected in workflow assertions.	Add rollback and restart tests; document SQLite limitations.
External integrations	Simulated outcomes only.	Contract tests for payment and supplier adapters before production use.
Security and performance	Not covered by this suite.	Add auth, input validation, load and penetration test plans.

8. Conclusion

The executed suite passes all six repository workflow tests. It provides useful evidence that TravelEase preserves inventory and payment consistency across successful, failed and cancelled bookings. The result is a strong academic baseline, but it should not be read as production certification because live integrations, security testing, load testing and long-term persistence are outside the current scope.

References

- GitHub repository: <https://github.com/bharath-541/travelease-system-design>
- Repository tests: `tests/test_workflows.py`; executed command: `python -m pytest -q`.
- Repository screenshots: `docs/screenshots/home.png`, `failure.png` and `confirmation.png`.